



MANUTENÇÃO DE SOFTWARE · 2026.1

AWS SAM Transform

Conversor de infraestrutura serverless

Luis Eduardo Vieira de Oliveira

Francisco Kauan Pereira Cavalcante

Antes de tudo: o que é “serverless”?

Servidores continuam existindo, só que quem cuida deles é a nuvem



Modelo tradicional

- Você provisiona servidores que ficam ligados o tempo todo
- Você escala, atualiza e protege essas máquinas manualmente
- Custo existe mesmo sem nenhuma requisição chegando



Modelo serverless

- Você escreve a função (ex.: AWS Lambda); a nuvem cuida do resto
- A infraestrutura nasce e escala sob demanda, automaticamente
- Você paga só pelo tempo em que o código realmente roda

AWS SAM Transform: o “tradutor” do serverless

Uma macro do *CloudFormation*(ferramenta de IaC) que *expande atalhos serverless* em infraestrutura completa



Sem o SAM, criar uma Lambda com API exige escrever na mão a função, a Role IAM, a REST API, métodos e permissões — **o SAM faz tudo isso “por baixo dos panos” a partir de poucas linhas.**

Por que importa: reduz boilerplate, embute boas práticas de segurança (IAM) e acelera o ciclo de desenvolvimento serverless.

Os arquivos mais complexos não são acaso

O ranking de criticidade do Pylint espelha a arquitetura do tradutor

Top arquivos mais críticos (Pylint)

1	sam_resources.py	411
2	push.py (eventos API)	211
3	api_generator.py	172
4	swagger.py	144

ocorrências de problemas de código



São exatamente as classes que “traduzem”

SamFunction.to_cloudformation é o método que pega a declaração curta de uma função Lambda e expande ela em Lambda + Role do IAM + Versões + Alias, tudo de uma vez.

api_generator e **swagger.py** constroem o REST/HTTP API e o domínio inteiros a partir de poucos eventos declarados.

Três code smells, 32 ocorrências corrigidas

Apoiados pela IA (DeepSeek V4 Flash, via OpenCode), sempre validando com a suíte de testes



7

ocorrências

too-many-statements

Métodos muito longos misturando responsabilidades; resolvido com Extract Method.



11⁺⁴

ocorrências

too-many-instance-attributes

Classes “inchadas”; atributos relacionados agrupados em dataclasses.



14

ocorrências

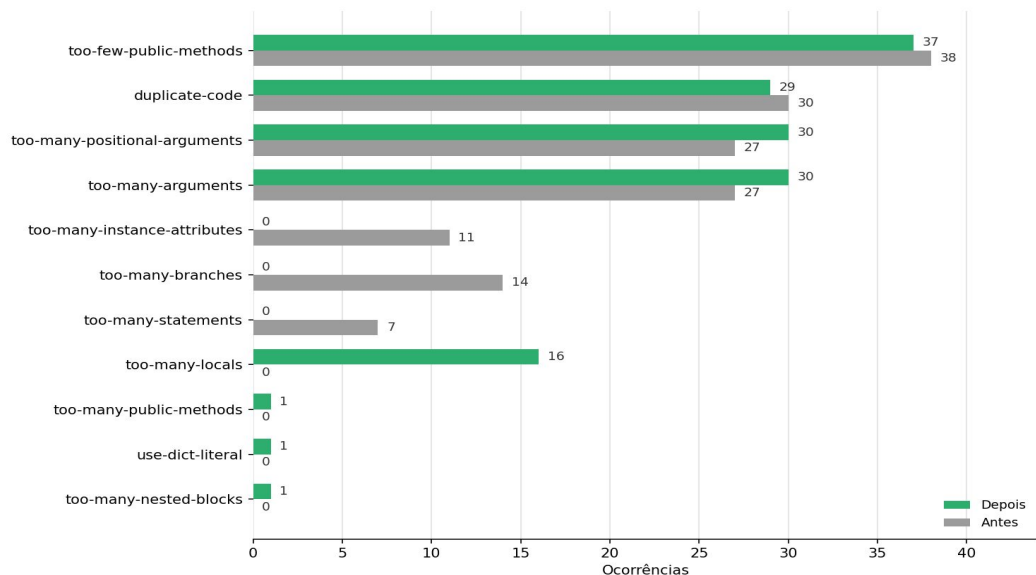
too-many-branches

Excesso de condicionais; lógica isolada em sub-métodos especializados.

Por ser um conversor de sintaxe, muitos smells nascem do próprio domínio — mapear vários formatos de evento em poucos métodos tende a gerar branches e parâmetros em excesso.

O que melhorou (e o que não)

Os smells atacados praticamente sumiram; outros aumentaram como efeito colateral



Score do projeto

7.3 → 7.40
/ 10

Total de problemas

3.087 → 3020
- 67 ocorrências no total

too-many-arguments subiu de 27 para 31: extrair métodos às vezes exige passar mais parâmetros entre eles.

Antes e depois, lado a lado

Complexidade caiu nos pontos críticos; testes pedem mais atenção

CC máxima — `_add_swagger_integration`

32 → **18**

Índice de manutenibilidade (pior
arquivo)

0.0 → **0.0**

Linhas de código (LOC)

27.358 →
27.779

Testes aprovados

4.256 → **4.190**

Testes falhando

283 → **349**

Cobertura de testes

95% → **95%**

O que levamos dessa experiência

Primeira contribuição em um projeto open source de grande porte



IA acelera, não substitui

A LLM aplicou bem padrões como Extract Method e dataclasses, em minutos por sessão — mas regressões nos testes mostram que a auditoria humana continua essencial.



Refatorar é fazer trade-off

Reduzir branches e statements às vezes aumenta o número de parâmetros ou de linhas totais: uma métrica melhora, outra pode piorar.



Testar a cada passo, não só no fim

Testes falhando subiram de 283 para 349. Rodar a suíte a cada refatoração, e não só ao final, teria pego essas regressões antes.



Obrigado

AWS SAM Transform — refatoração guiada por métricas e IA

Fork

github.com/LuisDallas1/serverless-application-model

Projeto original

github.com/aws/serverless-application-model

Pull Requests

github.com/LuisDallas1/serverless-application-model/pulls